

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра электронных вычислительных машин

Лабораторная работа №4
«Организация параллельной обработки запросов»
Вариант № 12

Выполнили:
студенты группы 250501
Снитко Д.А.
Шпаковский П.И.
Щербаков Д.И.

Проверил:
Андриевский Е.С.

Минск 2025

СОДЕРЖАНИЕ

ЦЕЛЬ И ЗАДАЧИ	2
1 ДЕМОНСТРАЦИЯ РАБОТЫ	4
1.1 Запуск сервера и подключение клиентов	4
1.2 Параллельная обработка запросов клиентов	4
2 ОПИСАНИЕ ПРОГРАММНЫХ МОДУЛЕЙ	6
2.1 Сервер	6
2.2 Клиент	7
ЗАКЛЮЧЕНИЕ	9
ПРИЛОЖЕНИЕ А.....	10

ЦЕЛЬ И ЗАДАЧИ

Цель лабораторной работы – изучить методы организации параллельной обработки запросов в сетевых приложениях.

Задачи лабораторной работы:

1. Модифицировать сервер для параллельного обслуживания клиентов с использованием пула процессов.
2. Реализовать обработку нескольких клиентов по протоколу TCP.
3. Провести тестирование сервера и продемонстрировать его работу.

1 ДЕМОНСТРАЦИЯ РАБОТЫ

1.1 Запуск сервера и подключение клиентов

Для запуска сервера необходимо выполнить команду:

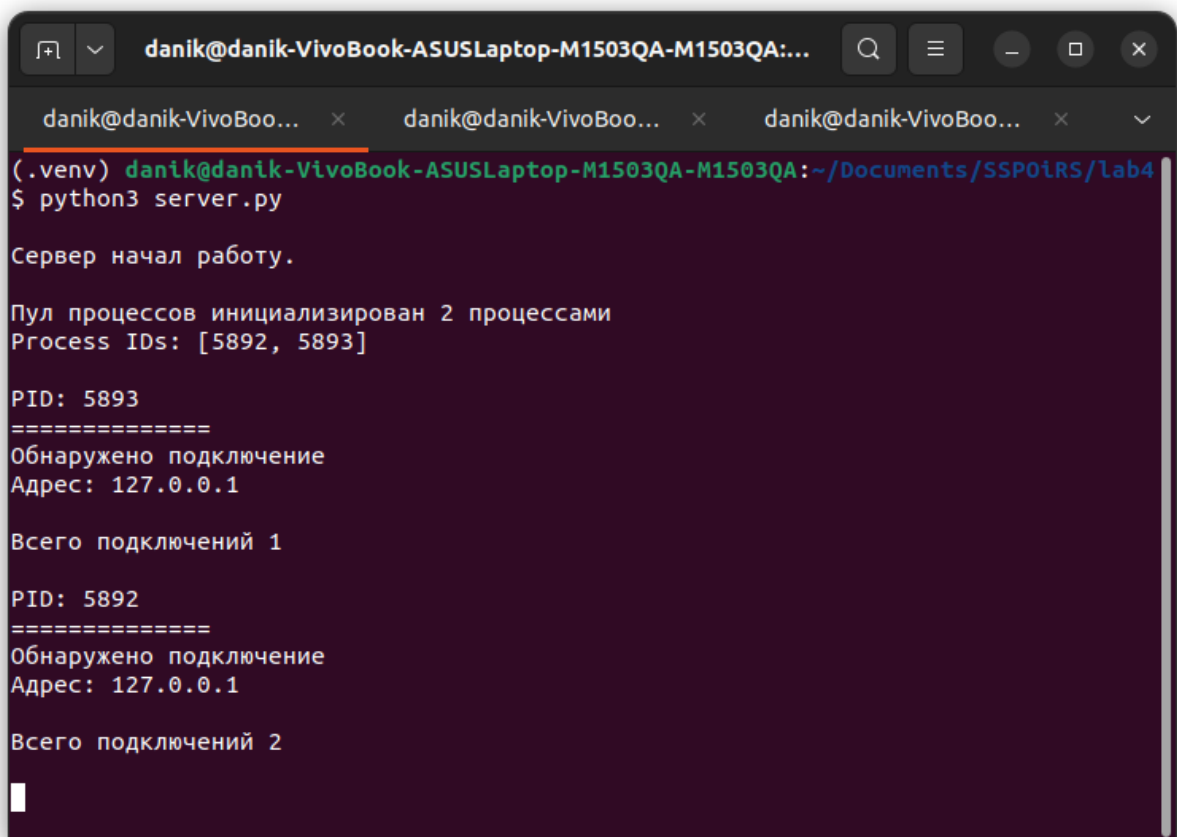
```
python3 server.py
```

Каждый клиент запускается отдельным процессом, который подключается к серверу.

Для запуска и подключения клиента необходимо выполнить команду:

```
python3 client.py
```

На рисунке 1.1 представлен результат запуска сервера и подключения двух клиентов:



```
danik@danik-VivoBook-ASUSLaptop-M1503QA-M1503QA:~$ python3 server.py
Сервер начал работу.
Пул процессов инициализирован 2 процессами
Process IDs: [5892, 5893]

PID: 5893
=====
Обнаружено подключение
Адрес: 127.0.0.1

Всего подключений 1

PID: 5892
=====
Обнаружено подключение
Адрес: 127.0.0.1

Всего подключений 2
```

Рисунок 1.1 – Запуск сервера и подключение клиентов

1.2 Параллельная обработка запросов клиентов

На рисунках 1.2 и 1.3 результат параллельного выполнения команд.

```
danik@danik-VivoBook-ASUSLaptop-M1503QA-M1503QA:...
Команда пользователя 127.0.0.1
Содержание: TIME
Ответ: Время сервера: Sat Mar 29 18:59:52 2025

PID: 5892
=====
Команда пользователя 127.0.0.1
Содержание: ECHO QWWE
Ответ: QWWE

PID: 5893
=====
Начата передача файла utserver.tar.gz
Получатель: 127.0.0.1

Начало загрузки. Оффсет: 0
Загрузка завершена.
PID: 5893
=====
Команда пользователя 127.0.0.1
Содержание: DOWNLOAD utserver.tar.gz
Ответ: Файл передан успешно.
```

Рисунок 1.2 – Результат выполнения команд

```
Получатель: 127.0.0.1

Начало загрузки. Оффсет: 0
Загрузка завершена.
PID: 5893
=====
Команда пользователя 127.0.0.1
Содержание: DOWNLOAD utserver.tar.gz
Ответ: Файл передан успешно.

PID: 5892
=====
Начат приём файла utserver.tar.gz
Отправитель: 127.0.0.1

Начало загрузки. Оффсет: 0
Загрузка завершена.
PID: 5892
=====
Команда пользователя 127.0.0.1
Содержание: UPLOAD utserver.tar.gz
Ответ: Файл загружен успешно.
```

Рисунок 1.3 – Результат выполнения команд

2 ОПИСАНИЕ ПРОГРАММНЫХ МОДУЛЕЙ

Код программы представлен в приложении А.

2.1 Сервер

Файл `server.py`

Управляет пулом процессов-воркеров: создаёт минимальное количество при старте, динамически масштабирует их число под нагрузку (добавляет при дефиците, удаляет при простоях). Обработывает сигналы от воркеров через `pipe`-каналы (новые подключения, завершения, таймауты), обеспечивает `accept()` через общий сокет.

Основные функции:

1. `createWorker` - Запускает дочерний процесс (`worker`), создает `pipe` для связи, передает воркеру дескриптор сокета, таймаут и `pipe`. Обновляет списки `PID` и каналов.
2. `destroyWorker` - Завершает процесс по указанному `PID`, удаляет его из списков активных воркеров и закрывает связанный `pipe`.
3. `initProcessPool` - Создает начальный пул из `PROC_NMIN` воркеров при старте сервера.
4. `destroyProcessPool` - Уничтожает все активные воркеры и закрывает их `pipe`-каналы при завершении работы сервера.
5. `printWorkersInfo` - Выводит текущее количество воркеров и их `PID` в консоль.
6. `addDynamicWorker` - Добавляет новый воркер при достижении условий (нагрузка = текущее число воркеров, `PROC_NMAX` не превышен).
7. `removeDynamicWorker` - Удаляет воркер по `PID` при его неактивности, если общее число воркеров больше `PROC_NMIN`.
8. `setOptions` - Включает опцию `SO_REUSEADDR` для сокета, позволяя повторно использовать адрес после перезапуска.
9. `makeFDInheritable` - Снимает флаг `FD_CLOEXEC` с файлового дескриптора, чтобы дочерние процессы могли его унаследовать.
10. `handleWorkerTimeout` - Проверяет, можно ли удалить воркер по таймауту (если количество воркеров больше минимального), и вызывает его уничтожение.

Файл `server_worker.py`

Обработывает отдельные клиентские подключения: выполняет команды (эхо, время, загрузка/выгрузка файлов), поддерживает докачку, логирует действия. При бездействии уведомляет основной сервер о таймауте, корректно завершается по сигналу `SIGTERM`. Основные функции:

1. `setOptions` - Настраивает TCP-сокет: включает `SO_KEEPALIVE`, таймауты и интервалы для поддержания соединения.

2. `Download` - Проверяет существование файла, отправляет клиенту статус (найден/не найден), запускает передачу файла через `downloadFile`.
3. `downloadFile` - Передаёт файл клиенту с поддержкой докачки. Получает оффсет от клиента, отправляет данные частями по `FRAME_SIZE`.
4. `Upload` - Принимает файл от клиента, проверяет его наличие у клиента, запускает сохранение через `uploadFile`.
5. `uploadFile` - Сохраняет данные от клиента в файл с докачкой. Определяет режим записи (`ab` или `wb`), обновляет файл до полного размера.
6. `Echo` - Возвращает клиенту переданный аргумент (эхо-ответ).
7. `_time` - Отправляет клиенту текущее время сервера.
8. `_exit` - Закрывает соединение с клиентом.
9. `handleCommand` - Разбирает команду клиента и вызывает соответствующую функцию-обработчик (`echo`, `time`, `download`, `upload`, `exit`).
10. `printStartFileLoading` - Выводит информацию о начале передачи файла (отправка/приём) с указанием IP-адреса клиента.
11. `printLog` - Логирует команду клиента и ответ сервера.
12. `wrapPrint` - Форматирует вывод в консоль с добавлением PID процесса и разделителей.
13. `termHandler` - Обрабатывает сигнал `SIGTERM`: закрывает сокет и `pipe`, завершает процесс.
14. `timeoutHandler` - Отправляет через `pipe` сигнал "T" при таймауте бездействия.

2.2 Клиент

Клиентская программа реализует подключение к пулу воркеров, отправляет команды и файлы, получает ответы. При разрыве связи предлагает переподключение, сохраняя функциональность. Основные функции:

1. `setOptions`
Настраивает TCP-сокет: включает `SO_KEEPALIVE`, задаёт интервалы проверки активности соединения.
2. `Connect` - Устанавливает соединение с сервером по адресу `HOST_ADDRESS:HOST_PORT`, применяет настройки сокета.
3. `reconnectPrompt` - Запрашивает у пользователя повторное подключение при обрыве связи (ответ `y/n`).
4. `Upload` - Отправляет файл на сервер: проверяет его наличие, передаёт размер и данные с докачкой. Возвращает статус операции.
5. `uploadFile` - Реализует передачу файла частями (`FRAME_SIZE`), поддерживает возобновление с последнего оффсета. Возвращает оффсет и размер файла.

6. `Download` - Запрашивает файл с сервера, проверяет его наличие, запускает загрузку через `downloadFile`. Рассчитывает скорость передачи.
7. `downloadFile` - Принимает файл от сервера, сохраняет с докачкой (режим `ab` или `wb`). Обновляет файл до полного размера.
8. `Exit` - Устанавливает флаг `exitFlag` для завершения работы клиента.
9. `otherCommand` - Обработывает команды, не требующие специальной логики (например, `echo`, `time`), возвращая ответ сервера.
10. `handleCommand` - Разбирает ввод пользователя, вызывает соответствующий обработчик (`upload`, `download`, `exit`) или перенаправляет запрос серверу.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы был разработан многопроцессный ТСР-сервер, эффективно управляющий параллельными клиентскими подключениями. Сервер динамически адаптируется к нагрузке, автоматически увеличивая пул рабочих процессов при росте запросов и освобождая ресурсы в периоды простоя, что обеспечивает стабильную работу даже при высокой активности клиентов. Реализованная модель с общим сокетом и атомарным `accept()` позволила избежать конфликтов при распределении подключений между процессами, а механизм обмена статусами через `pipe`-каналы гарантировал синхронизацию данных о состоянии системы. Поддержка докачки файлов, обработка базовых команд (`echo`, `time`) и расчёт скорости передачи данных продемонстрировали надёжность взаимодействия с клиентами.

ПРИЛОЖЕНИЕ А

(обязательное)

Код программы

Файл server.py

```
import socket
import os
import fcntl
import select

BIND_ADDRESS = "127.0.0.1"
BIND_PORT = 54321
FRAME_SIZE = 1024

SIGTERM = 15

PROC_NMIN = 2
PROC_NMAX = 5
WORKER_TIMEOUT = 10

def createWorker(serverSocket, timeout):
    readfd, writefd = os.pipe()
    makeFDInheritable(writefd)
    pid = os.fork()
    if pid == 0:
        os.execv('/home/danik/Documents/SSPOiRS/lab4/.venv/bin/python',
                 ['python3', os.path.realpath("./server_worker.py"),
                  str(serverSocket.fileno()), str(timeout),
                  str(writefd)])

    global workerCount
    workerCount += 1
    workersPID.append(pid)
    workerPipes.append(readfd)

    return pid

def destroyWorker(pid):
    os.kill(pid, SIGTERM)

    del workerPipes[(workersPID.index(pid))]
    workersPID.remove(pid)
    global workerCount
    workerCount -= 1

def initProcessPool(serverSocket):
    for i in range(PROC_NMIN):
        createWorker(serverSocket, WORKER_TIMEOUT)

    print(f"Пул процессов инициализирован {workerCount} процессами")
    print(f"Process IDs: {workersPID}\n")

def destroyProcessPool():
    for pid in workersPID:
        destroyWorker(pid)
    for pipe in workerPipes:
```

```

        os.close(pipe)

def printWorkersInfo():
    print(f"Текущее кол-во процессов: {workerCount}")
    print(f"Process IDs: {workersPID}\n")

def addDynamicWorker(serverSocket):
    pid = createWorker(serverSocket, WORKER_TIMEOUT)
    global skipSelect
    skipSelect = True
    print(f"Выделен новый процесс (PID: {pid})")
    printWorkersInfo()

def removeDynamicWorker(pid):
    destroyWorker(pid)
    print(f"Процесс (PID: {pid}) уничтожен по причине неактивности.")
    printWorkersInfo()

def setOptions(serverSocket):
    serverSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    return serverSocket

def makeFDInheritable(fd):
    flags = fcntl.fcntl(fd, fcntl.F_GETFD)
    fcntl.fcntl(fd, fcntl.F_SETFD, flags & ~fcntl.FD_CLOEXEC)

def handleWorkerTimeout(pid):
    if workerCount > PROC_NMIN:
        removeDynamicWorker(pid)

try:
    workerCount = 0
    connCount = 0
    workersPID = []
    workerPipes = []

    serverSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    serverSocket = setOptions(serverSocket)
    serverSocket.bind((BIND_ADDRESS, BIND_PORT))
    serverSocket.listen()

    makeFDInheritable(serverSocket.fileno())

    print("\nСервер начал работу.\n")

    initProcessPool(serverSocket)

    while True:
        pipesToRead, _, _ = select.select(workerPipes, [], [], 0.1)
        for pipe in pipesToRead:
            status = os.read(pipe, 1).decode()
            if status == "A":
                connCount += 1
                print(f"Всего подключений {connCount}\n")
            if status == "F":
                connCount -= 1
                print(f"Всего подключений {connCount}\n")

```

```

        if status == "T":
            handleWorkerTimeout(workersPID[workerPipes.index(pipe)])

    if workerCount >= PROC_NMIN and workerCount < PROC_NMAX and workerCount
    == connCount:
        newConn, _, _ = select.select([serverSocket], [], [], 0.1)
        if newConn:
            addDynamicWorker(serverSocket)

except KeyboardInterrupt:
    print("\nЗавершение работы.\n")
finally:
    destroyProcessPool()

```

server_worker.py

```

import socket
import time
import os
import sys
import signal

OPT_INTERVAL = 5
OPT_COUNT = 2
FRAME_SIZE = 1024

def setOptions(clientSocket):
    clientSocket.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
    clientSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    clientSocket.setsockopt(socket.SOL_TCP, socket.TCP_USER_TIMEOUT, 30000)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIIDLE,
    OPT_INTERVAL)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIIDLE,
    OPT_INTERVAL)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPCNT,
    OPT_COUNT)

    return clientSocket

def download(fileName):
    """Обрабатывает команду загрузки файла с сервера."""
    if not os.path.exists(fileName):
        clientSocket.send("0".encode("utf-8")) # Файл не найден
        return f"Файл \"{fileName}\" не найден."

    clientSocket.send("1".encode("utf-8")) # Файл существует

    printStartFileLoading(fileName, False)

    downloadFile(fileName)
    time.sleep(2)
    return "Файл передан успешно."

def downloadFile(fileName):
    """Передаёт файл клиенту с возможностью докачки."""
    fileSize = os.path.getsize(fileName)

    # Получаем оффсет от клиента (10 байт - целое число)
    offsetData = clientSocket.recv(10)
    if not offsetData:

```

```

        return

offset = int(offsetData.decode("utf-8").strip())

# Отправляем заголовок с полным размером файла (10 байт - целое число)
clientSocket.send(f"{fileSize:010}".encode("utf-8"))

# Начинаем отправку файла с указанного оффсета
with open(fileName, 'rb') as file:
    file.seek(offset)
    if(offset != 0):
        print(f"Продолжение загрузки с оффсета: {offset}")
    else:
        print(f"Начало загрузки. Оффсет: {offset}")

    while True:
        data = file.read(FRAME_SIZE)
        if not data:
            break
        try:
            clientSocket.send(data)
        except socket.error:
            print("Загрузка прервана.")
            return # Прерывание загрузки из-за ошибки соединения

print("Загрузка завершена.")

def upload(fileName):
    clientHasFile = clientSocket.recv(1).decode("utf-8")
    if clientHasFile == "0":
        return clientSocket.recv(FRAME_SIZE).decode("utf-8")

    printStartFileLoading(fileName, True)

    uploadFile(fileName)
    return "Файл загружен успешно."

def uploadFile(fileName):
    mode = 'ab' if os.path.exists(fileName) else 'wb+'

    with open(fileName, mode) as file:
        fileSize = int(clientSocket.recv(FRAME_SIZE).decode("utf-8"))
        clientSocket.send(str(os.path.getsize(fileName)).encode("utf-8"))
        offset = os.path.getsize(fileName)
        if (offset != 0):
            print(f"Продолжение загрузки с оффсета: {offset}")
        else:
            print(f"Начало загрузки. Оффсет: {offset}")
        file.seek(0, os.SEEK_END)

        while fileSize > offset:
            try:
                data = clientSocket.recv(FRAME_SIZE)
                if not data:
                    break
                file.write(data)
                offset += len(data)
            except socket.error:
                print("Загрузка прервана.")
                return # Прерывание загрузки из-за ошибки соединения

    print("Загрузка завершена.")

```

```

def echo(data):
    if data:
        return data
    return "Команда 'echo' требует аргумент."

def _time():
    currentTime = time.asctime(time.localtime())
    return "Время сервера: " + currentTime

def _exit():
    global connected
    connected = False
    return "Отключение от сервера..."

def handleCommand(clientInput):
    command, argument = clientInput.partition(" ")[::2]

    match (command.lower()):
        case "echo":
            response = echo(argument)

        case "time":
            response = _time()

        case "download":
            response = download(argument)

        case "upload":
            response = upload(argument)

        case "exit":
            response = _exit()

        case _:
            response = "Такой команды не существует!"

    return response

def printStartFileLoading(fileName, dir):
    if dir:
        wrapPrint(f"Начат приём файла {fileName}\n"
                  f"Отправитель: {clientAddr[0]}")
    else:
        wrapPrint(f"Начата передача файла {fileName}\n"
                  f"Получатель: {clientAddr[0]}")

def printLog(clientInput, response):
    if clientInput:
        wrapPrint(f"Команда пользователя {clientAddr[0]}\n"
                  f"Содержание: {clientInput}\n"
                  f"Ответ: {response}")

def wrapPrint(*args, **kwargs):
    print(f"PID: {os.getpid()}")
    print("=====")
    print(*args, **kwargs)

```

```

print()

def termHandler(*args):
    if clientSocket and clientSocket.fileno() != -1:
        clientSocket.close()
    os.close(pipe)
    exit()

def timeoutHandler(*args):
    os.write(pipe, "T".encode())

try:
    serverSocket = socket.fromfd(int(sys.argv[1]), socket.AF_INET,
socket.SOCK_STREAM)
    clientSocket = None
    clientAddr = None
    connected = False

    timeout = int(sys.argv[2])
    pipe = int(sys.argv[3])

    signal.signal(signal.SIGTERM, termHandler)
    signal.signal(signal.SIGALRM, timeoutHandler)

    while True:
        signal.alarm(timeout)
        clientSocket, clientAddr = serverSocket.accept()
        signal.alarm(0)
        connected = True

        wrapPrint("Обнаружено подключение\n" + "Адрес:", clientAddr[0])
        os.write(pipe, "A".encode())

        clientSocket = setOptions(clientSocket)
        clientSocket.send("1".encode())

        while connected:
            try:
                clientInput = clientSocket.recv(FRAME_SIZE).decode()
                response = handleCommand(clientInput)
                clientSocket.send(response.encode())
                printLog(clientInput, response)

            except socket.error:
                wrapPrint(f"Обрыв соединения с {clientAddr[0]}")
                clientSocket.close()
                break
            os.write(pipe, "F".encode())

except KeyboardInterrupt:
    pass

```

Файл client.py

```

import socket
import os
import time
import select

```

```

HOST_ADDRESS = "127.0.0.1"
HOST_PORT = 54321
OPT_INTERVAL = 5
OPT_COUNT = 3
FRAME_SIZE = 4096

exitFlag = False

def setOptions(clientSocket):
    clientSocket.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
    # clientSocket.setsockopt(socket.SOL_TCP, socket.TCP_USER_TIMEOUT, 5000)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIIDLE,
OPT_INTERVAL)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIINTVL,
OPT_INTERVAL)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPCNT, OPT_COUNT)

    return clientSocket

def connect(clientSocket):
    clientSocket = setOptions(clientSocket)
    clientSocket.connect((HOST_ADDRESS, HOST_PORT))
    return clientSocket

def reconnectPrompt():
    print("Соединение разорвано.",
        "\nХотите попробовать восстановить соединение? [y/n]")

    answer = input()
    if (answer[0] == "Y") or (answer[0] == "y"):
        return True

    return False

def upload(filePath):
    if not os.path.exists(filePath):
        clientSocket.send("0".encode("utf-8"))
        clientSocket.send(("Файл \"" + filePath + "\" не найден.").encode("utf-
8"))
        return clientSocket.recv(FRAME_SIZE).decode("utf-8")

    clientSocket.send("1".encode("utf-8"))

    startTime = time.time()
    offset, fileSize = uploadFile(filePath)
    endTime = time.time()

    speed = "{:.2f}".format((fileSize - offset) / (endTime - startTime) / 1024)
    print("Скорость загрузки:", speed, "KB/c")
    return clientSocket.recv(FRAME_SIZE).decode("utf-8")

def uploadFile(filePath):
    with open(filePath, 'rb') as file:
        clientSocket.send(str(os.path.getsize(filePath)).encode("utf-8"))
        offset = int(clientSocket.recv(FRAME_SIZE).decode("utf-8"))

        if (offset != 0):
            print(f"Продолжение загрузки с оффсета: {offset}")
        else:
            print(f"Начало загрузки. Оффсет: {offset}")

        file.seek(offset, 0)

```



```

        while True:
            data = file.read(FRAME_SIZE)
            if not data:
                break
            clientSocket.send(data)

    file.close()
    return (offset, os.path.getsize(filePath))

def download(filePath, clientSocket):
    """Запрашивает у сервера файл и загружает его."""
    serverHasFile = clientSocket.recv(1).decode("utf-8")
    if serverHasFile == "0":
        return clientSocket.recv(FRAME_SIZE).decode("utf-8")

    startTime = time.time()
    offset, fileSize = downloadFile(filePath, clientSocket)
    endTime = time.time()

    speed = "{:.2f}".format(offset / (endTime - startTime) / 1024)
    print("Скорость загрузки:", speed, "КБ/с")

    return clientSocket.recv(FRAME_SIZE).decode("utf-8")

def downloadFile(filePath, clientSocket):
    """Принимает файл от сервера, поддерживает докачку."""
    mode = 'ab' if os.path.exists(filePath) else 'wb+'
    with open(filePath, mode) as file:
        offset = os.path.getsize(filePath)

        clientSocket.send(f"{offset:010}".encode("utf-8"))

        fileSizeData = clientSocket.recv(10)
        if not fileSizeData:
            return offset, offset

        fileSize = int(fileSizeData.decode("utf-8").strip())

        if (offset != 0):
            print(f"Продолжение загрузки с оффсета: {offset}")
        else:
            print(f"Начало загрузки. Оффсет: {offset}")

        startTime = time.time()
        while offset < fileSize:
            try:
                data = clientSocket.recv(FRAME_SIZE)
                if not data:
                    break
                file.write(data)
                offset += len(data)
            except socket.error:
                print("Загрузка прервана.")
                return offset, fileSize

    return offset, fileSize

def exit():
    global exitFlag
    exitFlag = True

def otherCommand(userInput):
    return clientSocket.recv(FRAME_SIZE).decode()

```

```

def handleCommand(userInput):
    command, argument = userInput.partition(" ")[::2]

    match (command.lower()):
        case "upload":
            response = upload(argument)

        case "download":
            response = download(argument, clientSocket)

        case _:
            response = otherCommand(userInput)

    if (userInput.lower() == "exit"):
        exit()

    return response

try:
    clientSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    clientSocket = connect(clientSocket)

    print("Ожидание подключения...")

    clientSocket.recv(1)

    print("Соединение установлено.\n")

    while not exitFlag:
        try:
            userInput = input("> ")
            while userInput == "":
                userInput = input("> ")
            clientSocket.send(userInput.encode())
            response = handleCommand(userInput)
            print(response, "\n")

        except socket.error:
            clientSocket.close()
            try:
                result = reconnectPrompt()
                if (result):
                    clientSocket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
                    clientSocket = connect(clientSocket)
                    print("Соединение восстановлено.\n")
                else:
                    exitFlag = True
            except socket.error:
                print("Не удалось восстановить подключение.")
                exitFlag = True

    except socket.error:
        print("Сервер недоступен.\n")
    except KeyboardInterrupt:
        print("\nЗавершение работы.\n")
    finally:
        clientSocket.close()

```